ParoCyber

DevSecOps

Assignment 1 – Linux User & Access Control : Scenario-Based DevSecOps Practical Assessment

Instructor: Samuel Nartey Otuafo

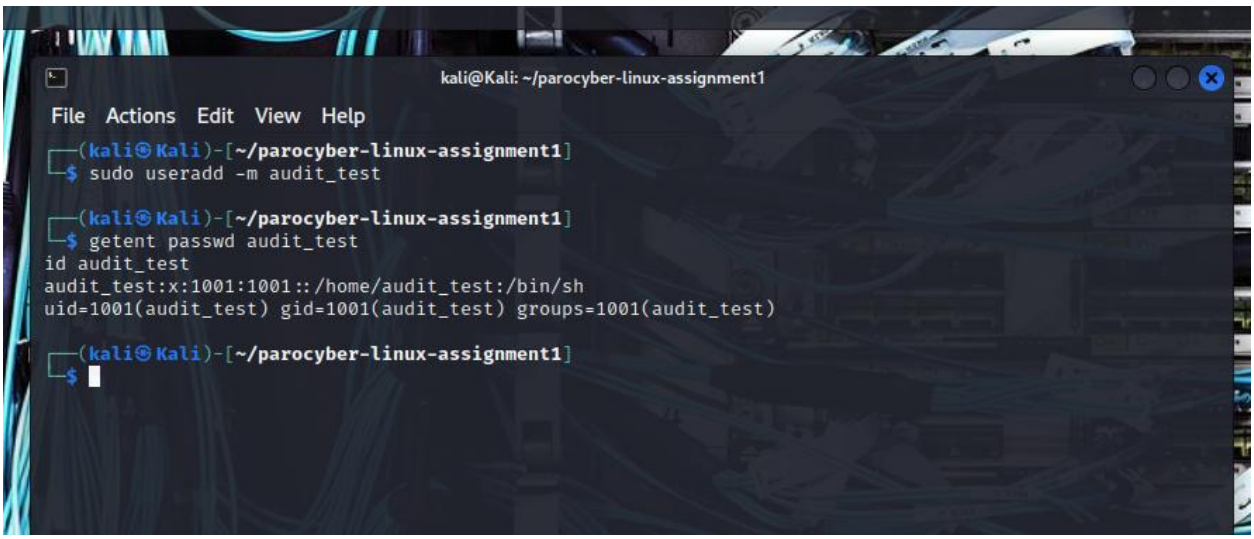
Date: 21-Feb-2026

Scenario 1: Password State Investigation & Remediation

Part A: Create a User With No Password

Commands Executed

sudo useradd -m audit_test
getent passwd audit_test
id audit_test



Explanation

- I created a new user **audit_test** using useradd -m, which also generated a home directory /home/audit_test.
- No password was set, which is the intended requirement.
- The getent passwd command confirms the user exists in /etc/passwd.
- The id command verifies the user's UID, GID, and assigned group.
- The presence of "x" in the password field indicates that password hashes are stored in /etc/shadow, not in /etc/passwd.

Part B: Identify Users Without Passwords (Discovery Required)

Files Identified

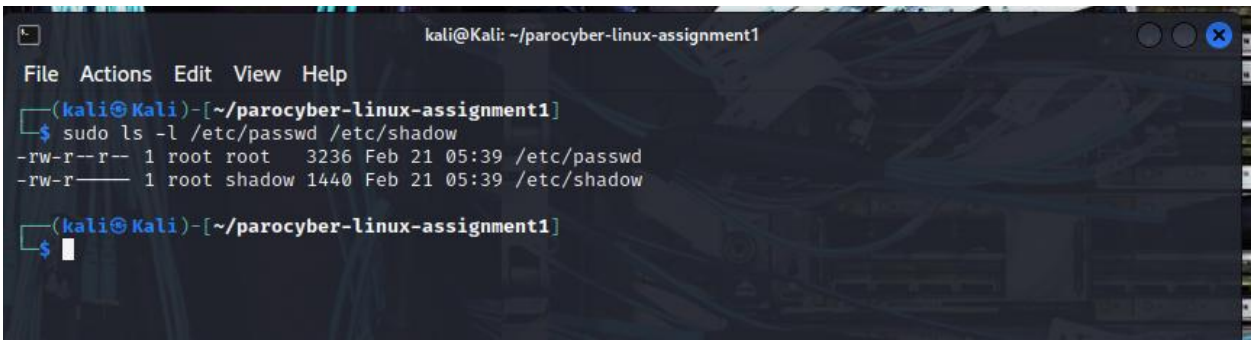
Linux stores user account information in:

- **/etc/passwd** : publicly readable

    - Contains username, UID/GID, shell, home directory
    - Password field shows x (placeholder)
- **/etc/shadow** : restricted (root-only)
    - Contains actual password hashes
    - Stores password lock state, last change, expiry rules

Evidence

sudo ls -l /etc/passwd /etc/shadow



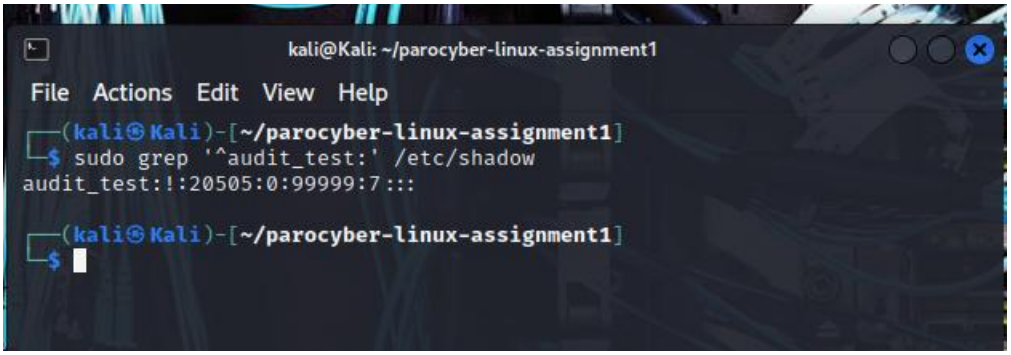
Confirms correct permissions:

- /etc/passwd → world-readable
- /etc/shadow → root-only access

Password State for audit_test

sudo grep '^audit_test:' /etc/shadow

Output:



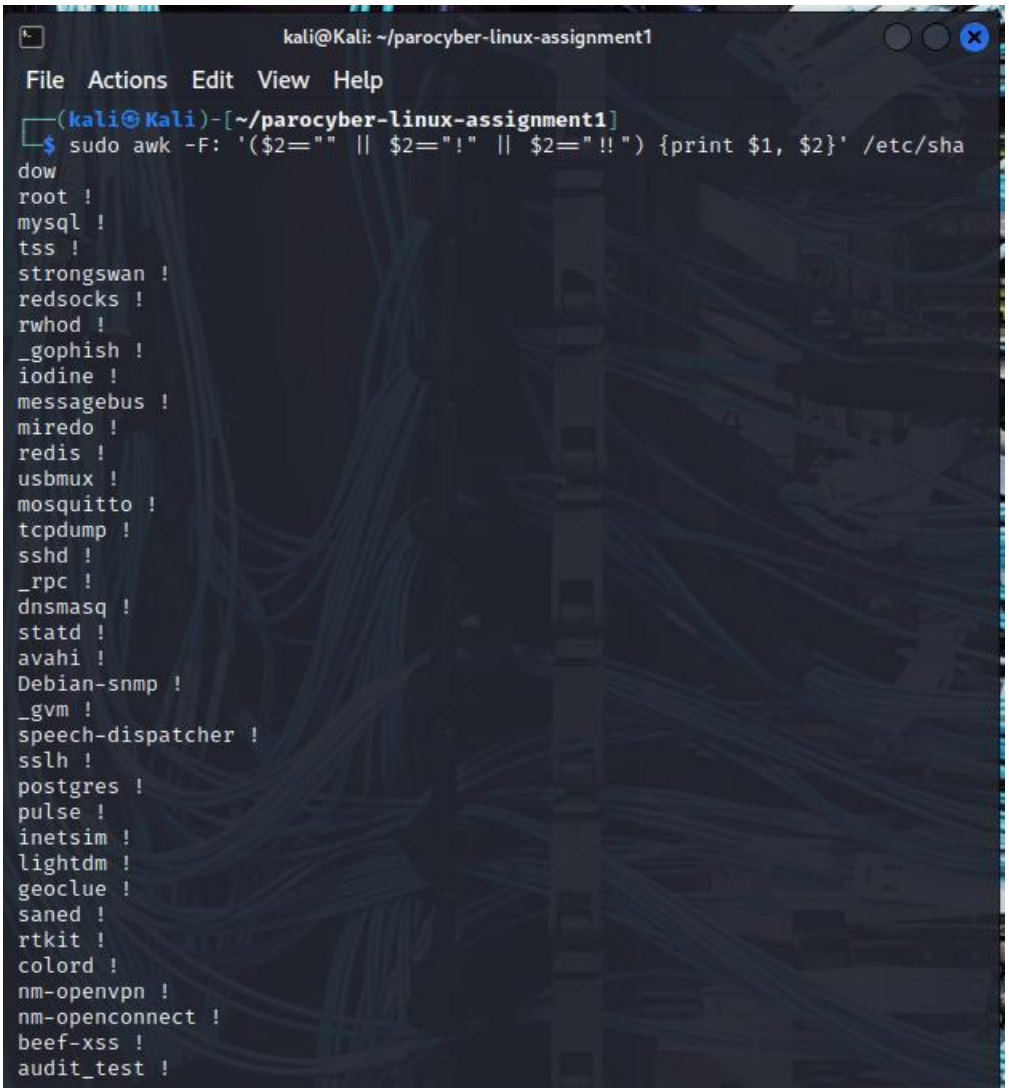
audit_test:!:20505:0:99999:7:::

Interpretation:

- Password field contains !
- Means **no set password / login disabled**
- Confirms user was created **without a password**, as required

Listing All Users Without Passwords

sudo awk -F: '($2==""" || $2=="!" || $2=="!!") {print $1, $2}' /etc/shadow

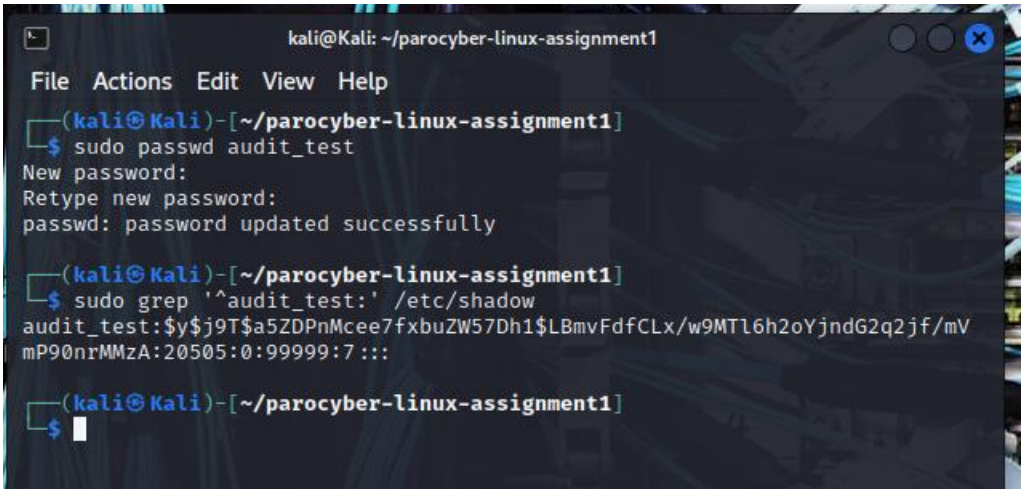


This revealed a list of accounts (mostly system accounts) that also have locked passwords. **audit_test** appears in the list, confirming its password state.

Part C: Remediation – Set a Password

Commands Executed

sudo passwd audit_test
sudo grep '^audit_test:' /etc/shadow



Explanation

The **passwd** command was used to set a new password for the **audit_test** user.
This action changed the user's password field in **/etc/shadow** from:

!

which indicates **no password / login disabled**

to a full hashed password string as:

yj9T$3S2DPnMcee7fxbuZW57Dh1$LBmvFdfCLx/w9MTL6h2oYjndG2q2jf/mVmP9OnrMMA2

This confirms that:

- The user now has a **valid password**
- The account is no longer in a vulnerable or unusable state
- Linux updated the password hash securely in **/etc/shadow**

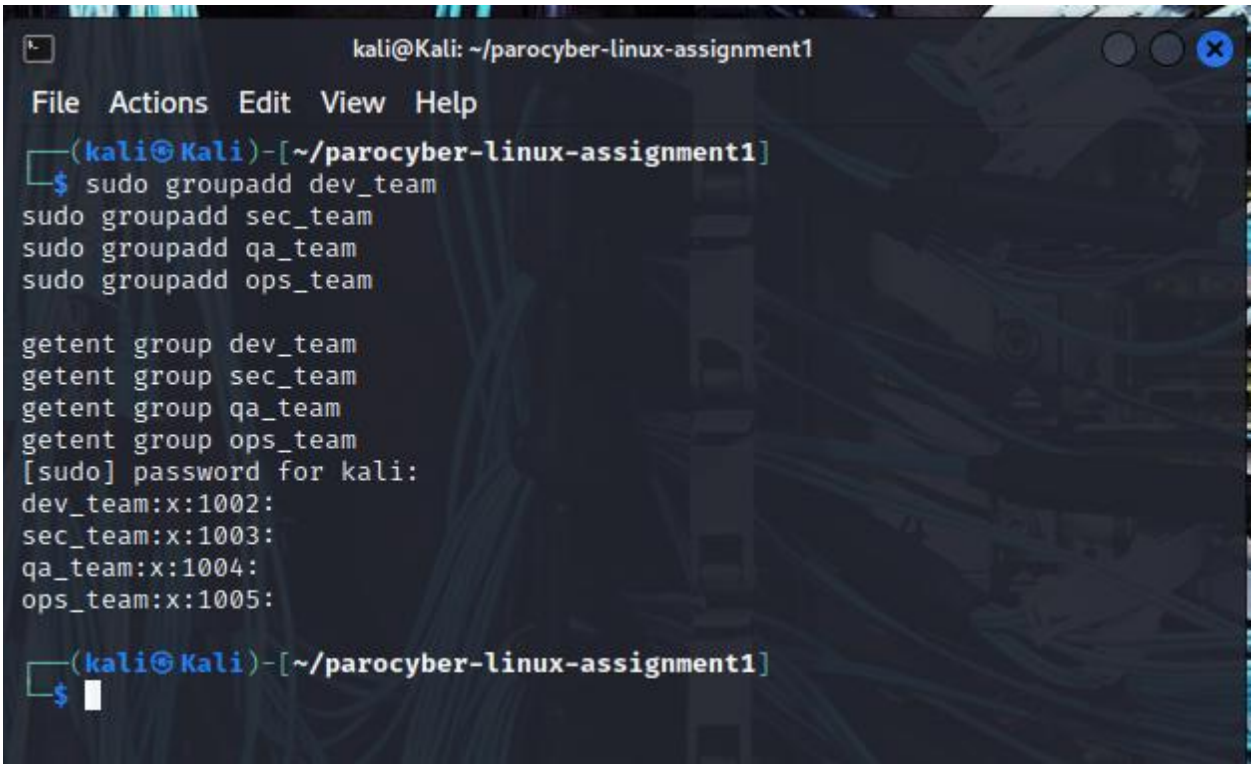
Scenario 2: DevSecOps User & Department Access Configuration

Part 0: Create Department Groups

Commands Executed

sudo groupadd dev_team
sudo groupadd sec_team
sudo groupadd qa_team
sudo groupadd ops_team

getent group dev_team
getent group sec_team
getent group qa_team
getent group ops_team



Explanation

Four Linux groups were created to represent the organizational departments required in the assignment:

- **dev_team**
- **sec_team**

- **qa_team**
- **ops_team**

The groupadd command creates each group inside /etc/group.
After creating them, the getent group command was used to verify that each group exists and has a valid GID (Group ID):

dev_team:x:1002:
sec_team:x:1003:
qa_team:x:1004:
ops_team:x:1005:

This confirms:

- All department groups were successfully created
- Each group has a unique GID

Part 1: Create DevSecOps Users

Commands Executed

```
sudo useradd -m lateef

sudo adduser yaa
sudo adduser alex
sudo adduser habiba
sudo adduser aminat
sudo adduser purity
sudo adduser arthur
sudo adduser paula

getent passwd lateef purity arthur paula yaa alex habiba aminat
```

```
kali@Kali: ~/parocyber-linux-assignment1
File  Actions  Edit  View  Help

┌──(kali㉿Kali)-[~/parocyber-linux-assignment1]
└─$ sudo useradd -m lateef
sudo adduser yaa
sudo adduser alex
sudo adduser habiba
sudo adduser aminat
sudo adduser purity
sudo adduser arthur
sudo adduser paula
info: Adding user `yaa' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `yaa' (1007) ...
info: Adding new user `yaa' (1007) with group `yaa (1007)' ...
info: Creating home directory `/home/yaa' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for yaa
Enter the new value, or press ENTER for the default
```

```
┌──(kali㉿Kali)-[~/parocyber-linux-assignment1]
└─$ getent passwd lateef purity arthur paula yaa alex habiba aminat
lateef:x:1002:1006::/home/lateef:/bin/sh
purity:x:1011:1011:,,,:/home/purity:/bin/bash
arthur:x:1012:1012:,,,:/home/arthur:/bin/bash
paula:x:1013:1013:,,,:/home/paula:/bin/bash
yaa:x:1007:1007:,,,:/home/yaa:/bin/bash
alex:x:1008:1008:,,,:/home/alex:/bin/bash
habiba:x:1009:1009:,,,:/home/habiba:/bin/bash
aminat:x:1010:1010:,,,:/home/aminat:/bin/bash

┌──(kali㉿Kali)-[~/parocyber-linux-assignment1]
└─$
```

Explanation

A set of DevSecOps-related user accounts was created to represent members of different departments.

- The useradd -m command was used to create **lateef** with a home directory /home/lateef.
- The adduser command (a more interactive, user-friendly wrapper) was used to create the remaining users:
    - **yaa, alex, habiba, aminat, purity, arthur, paula**
    - For each of these, adduser:
        - Chose a UID/GID,
        - Created a matching primary group,

- Created a home directory under /home/,
- Copied default files from /etc/skel,
- Prompted to set a password.

The final getent passwd command was used to verify that all required accounts exist and are correctly configured:

```
lateef:x:1002:1006::/home/lateef:/bin/sh
purity:x:1011:1011:,,,:/home/purity:/bin/bash
arthur:x:1012:1012:,,,:/home/arthur:/bin/bash
paula:x:1013:1013:,,,:/home/paula:/bin/bash
yaa:x:1007:1007:,,,:/home/yaa:/bin/bash
alex:x:1008:1008:,,,:/home/alex:/bin/bash
habiba:x:1009:1009:,,,:/home/habiba:/bin/bash
aminat:x:1010:1010:,,,:/home/aminat:/bin/bash
```

This confirms:

- Each user has a valid UID, primary GID, home directory, and shell.
- Both useradd and adduser were demonstrated, as required by the assignment.

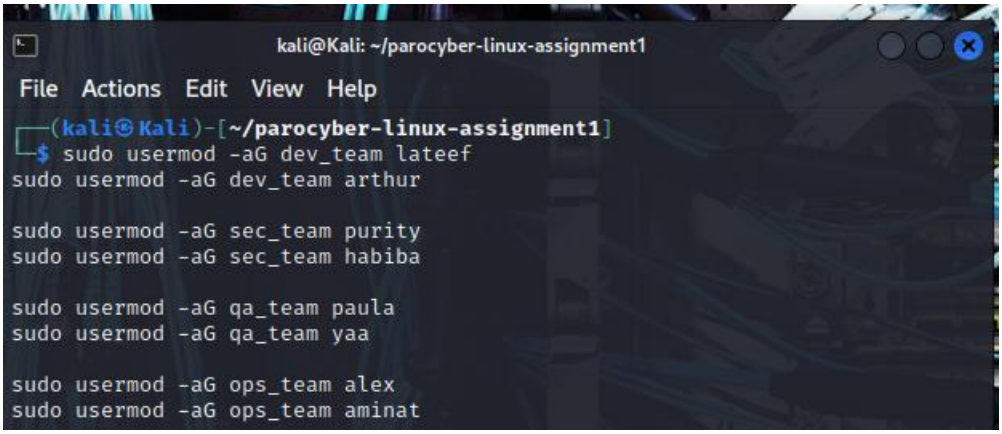
Part 2: Assign Users to Department Groups

Commands Executed

```
sudo usermod -aG dev_team lateef
sudo usermod -aG dev_team arthur

sudo usermod -aG sec_team purity
sudo usermod -aG sec_team habiba

sudo usermod -aG qa_team paula
sudo usermod -aG qa_team yaa

sudo usermod -aG ops_team alex
sudo usermod -aG ops_team aminat

getent group dev_team
getent group sec_team
```

getent group qa_team
getent group ops_team

id lateef
id arthur
id purity
id habiba
id paula
id yaa
id alex
id aminat



Explanation

The usermod -aG command was used to add each user to the correct **department (supplementary) group** without changing their primary group:

- lateef, arthur → dev_team
- purity, habiba → sec_team
- paula, yaa → qa_team
- alex, aminat → ops_team

After the assignments, getent group was used to confirm that the group entries in /etc/group correctly reflect the membership:

```
dev_team:x:1002:arthur,lateef
sec_team:x:1003:purity,habiba
qa_team:x:1004:paula,yaa
ops_team:x:1005:alex,aminat
```

This shows that each department group now lists its members, proving that **Linux records and tracks group membership in /etc/group**.

The **id** command was then run for each user to verify their **primary** and **supplementary** groups:

```
uid=1002(lateef) gid=1006(lateef) groups=1006(lateef),1002(dev_team)
uid=1012(arthur) gid=1012(arthur) groups=1012(arthur),100(users),1002(dev_team)
uid=1011(purity) gid=1011(purity) groups=1011(purity),100(users),1003(sec_team)
uid=1009(habiba) gid=1009(habiba) groups=1009(habiba),100(users),1003(sec_team)
uid=1013(paula) gid=1013(paula) groups=1013(paula),100(users),1004(qa_team)
uid=1007(yaa) gid=1007(yaa) groups=1007(yaa),100(users),1004(qa_team)
uid=1008(alex) gid=1008(alex) groups=1008(alex),100(users),1005(ops_team)
uid=1010(aminat) gid=1010(aminat) groups=1010(aminat),100(users),1005(ops_team)
```

From this output:

- Each user keeps their own primary group (their username).
- Each user is also in the correct **department group** (e.g., dev_team, sec_team, qa_team, ops_team).
- Some users also belong to the generic users group, which is normal on this system.

This demonstrates how **department-based grouping enforces access control**:
files and resources can be owned by these groups (e.g., dev_team, sec_team), and only

members of those groups will have the required read/write/execute permissions, supporting least privilege and clear separation between development, security, QA, and operations.

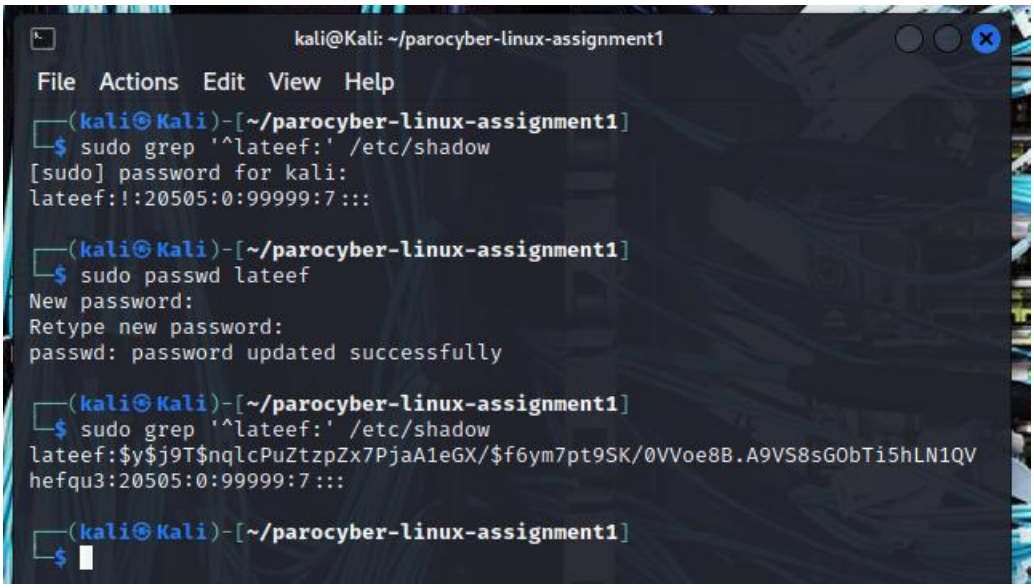
Part 3: Ensure One User Is Created Without a Password, Then Remediate

Commands Executed

sudo grep '^lateef:' /etc/shadow

sudo passwd lateef

sudo grep '^lateef:' /etc/shadow



Explanation

The user **lateef** was originally created using useradd -m, which does **not** assign a password by default.
As a result, the /etc/shadow entry for the user showed:

lateef:!

The ! symbol indicates:

- **No password set**
- **Login is locked**
- The account cannot be used for authentication

This satisfies the requirement to have **one user created without a password**.

To remediate this, the password was set using:

sudo passwd lateef

After setting the password, the password field in /etc/shadow updated to a long **hashed** value (e.g., y... or 6...), confirming the password was successfully created.

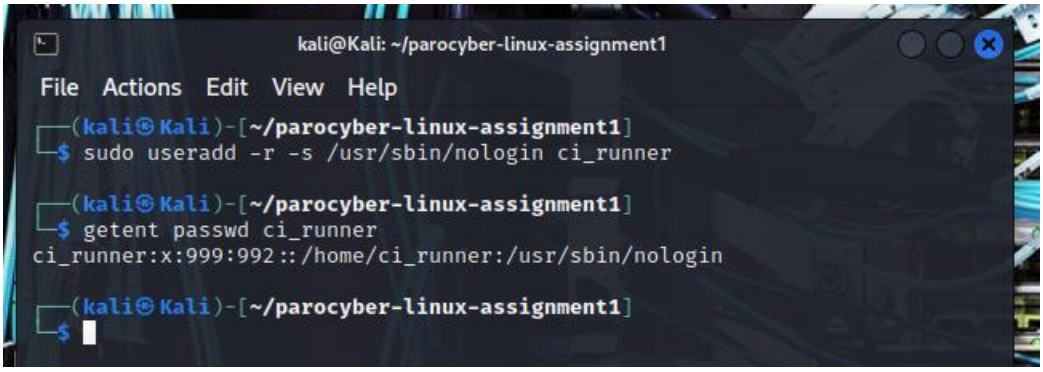
This meets the requirement to:

- Identify a passwordless user
- Remediate the issue
- Verify the change in /etc/shadow

Part 4: Configure the ci_runner Service Account With a Non-Login Shell

Commands Executed

sudo useradd -r -s /usr/sbin/nologin ci_runner
getent passwd ci_runner



Explanation

The ci_runner account is a **service account** used for CI/CD automation.
It must not allow human login.

The command:

sudo useradd -r -s /usr/sbin/nologin ci_runner

applies two important settings:

- **-r** creates a system account used by services

- **-s /usr/sbin/nologin** prevents login or interactive shell access

This ensures the account:

- Exists for automation tasks
- Cannot be used to log in
- Follows security best practices for service accounts

Running:

getent passwd ci_runner

confirms that:

- The account exists
- The shell is set to /usr/sbin/nologin

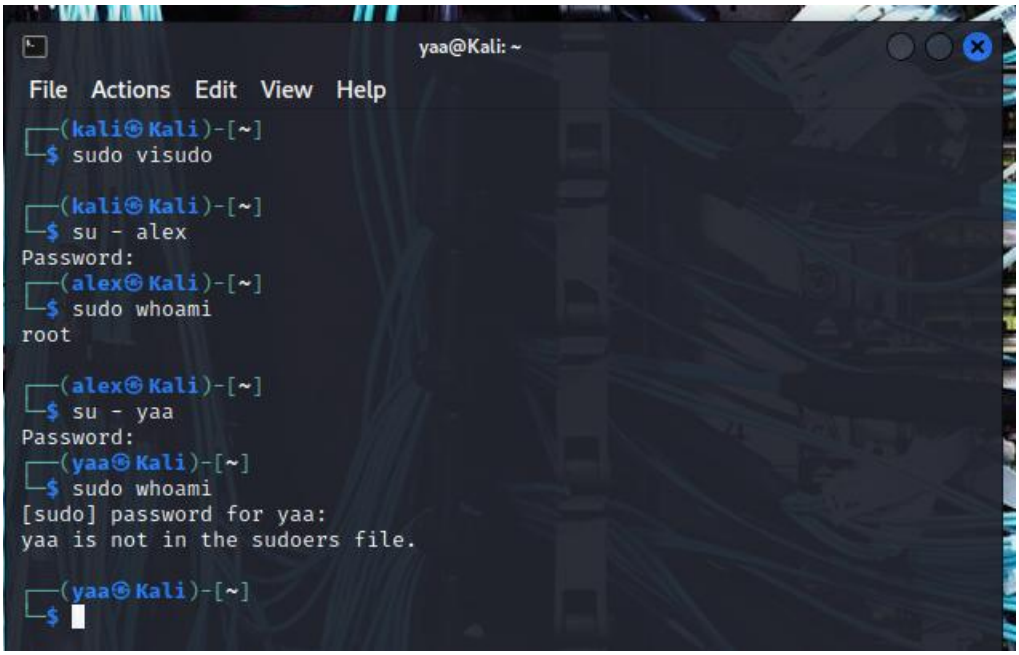
Part 5: Grant Sudo Access to ONE ops_team User Only

Commands Executed

sudo visudo

su - alex

sudo whoami



Explanation

Only **one** user from the **ops_team** group was granted privileged access to maintain strict **least privilege** and **audit traceability**.

The visudo command was used to safely edit the /etc/sudoers file.
The following rule was added:

alex ALL=(ALL:ALL) ALL

This means:

- **alex** can run any command with sudo
- This access is logged and auditable
- All other users remain restricted to their departmental privileges

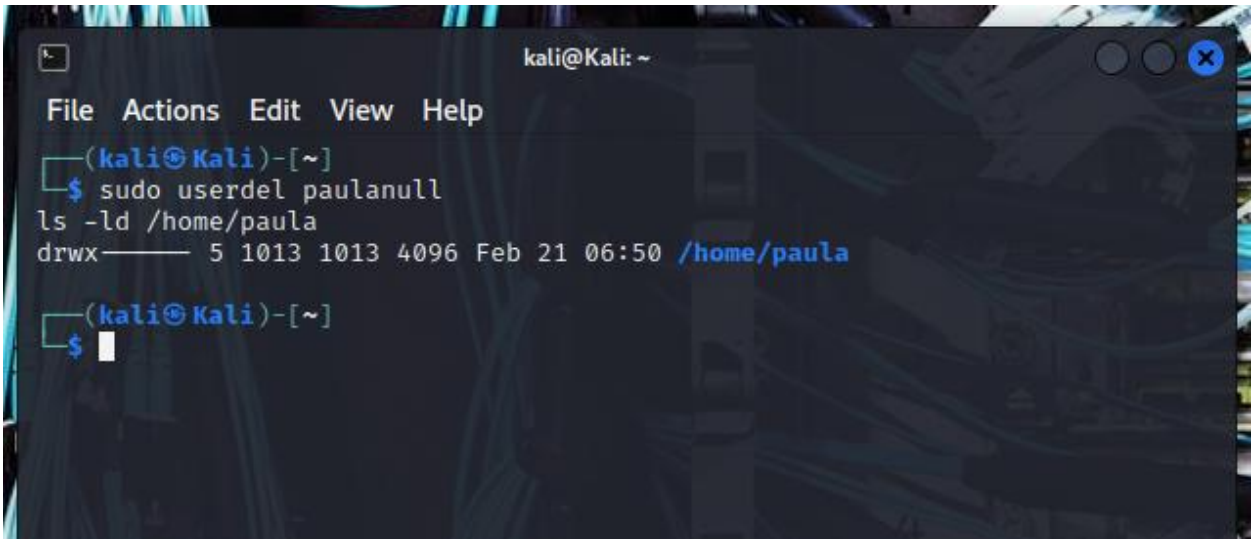
Verification steps:

- Logging in as **alex** and running sudo whoami produced root, confirming successful sudo permission.
- Testing sudo from a non-privileged user (e.g., yaa) correctly triggered a denial message, proving privilege boundaries are enforced.

Part 6: Remove One User While Preserving the Home Directory

Commands Executed

sudo userdel paula
ls -ld /home/paula



Explanation

The userdel command was used to delete the user account **paula** from the system.
The command was run **without** the -r flag:

sudo userdel paula

This removes:

- The user entry from /etc/passwd
- The user's group entry (if no longer needed)
- Authentication details from /etc/shadow

But **does NOT remove the home directory**.
This is important for:

- **Audit purposes**
- **Digital forensics**
- **Security investigations**
- **Retention of configuration or project files**

The command:

ls -ld /home/paula

returned:

drwx------ 5 1013 1013 4096 Feb 21 06:50 /home/paula

This confirms that:

- The home directory still exists
- Its ownership remains with the original UID/GID (1013)
- Files remain intact for later review or archiving

Part 7: Verification of Group Membership, Group Data, and Privilege Boundaries

Commands Executed

Verify group membership from /etc/group

getent group dev_team sec_team qa_team ops_team

Verify individual user group assignments

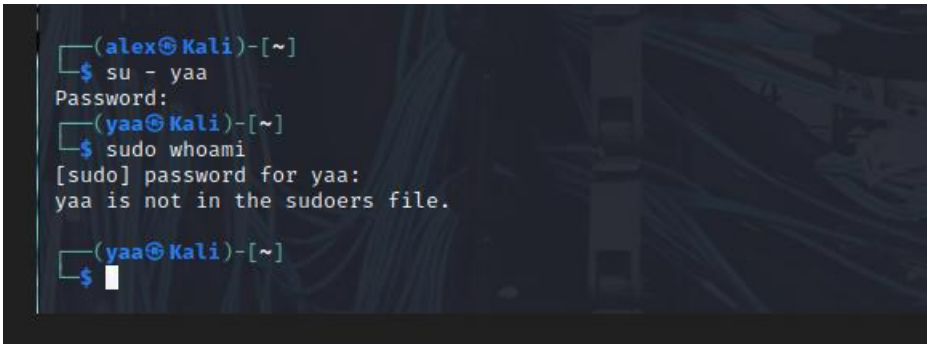
id alex
id aminat
id lateef

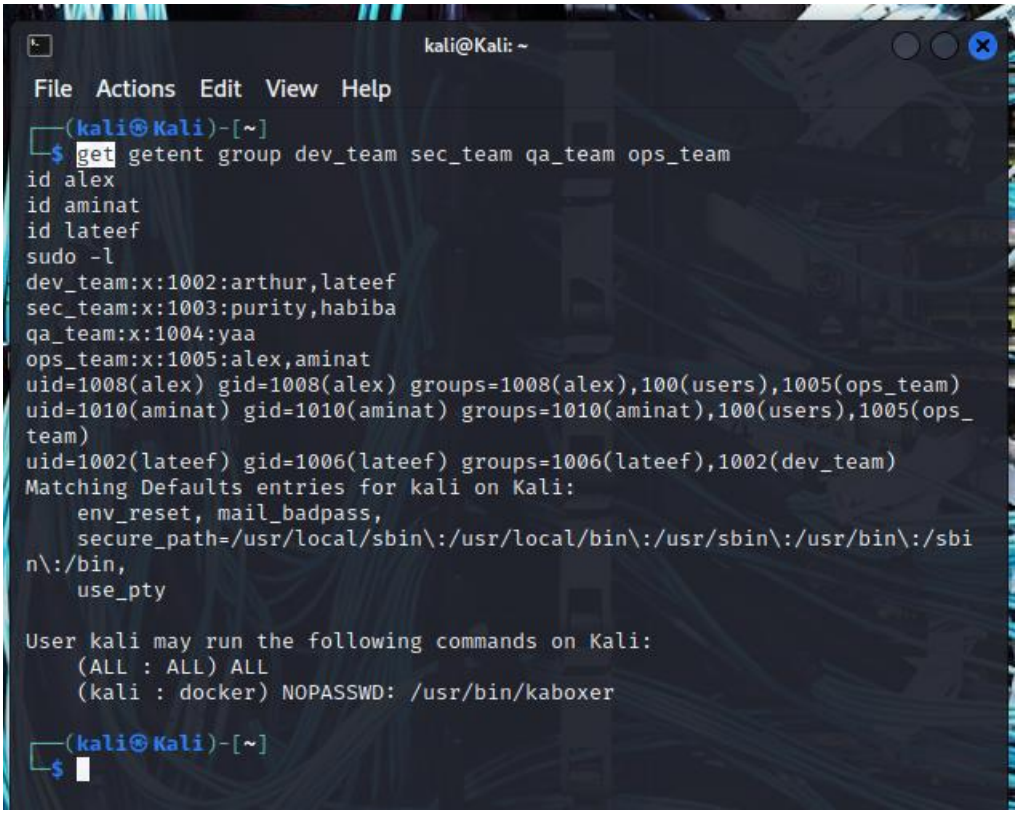
Check sudo privileges for the approved user (alex)

sudo -l

Attempt sudo with a non-privileged user (yaa)

su - yaa
sudo whoami





Explanation

This step verifies that:

1. **Department-based groups exist**,
2. **Users are correctly assigned**, and
3. **Privilege boundaries are strictly enforced**.

1. Group Membership Validation

The output of:

getent group dev_team sec_team qa_team ops_team

returned:

dev_team:x:1002:arthur,lateef
sec_team:x:1003:purity,habiba
qa_team:x:1004:yaa
ops_team:x:1005:alex,aminat

This proves that:

- Linux stores and tracks group membership in /etc/group
- Each user is correctly mapped to their respective department

2. User Group Verification

Running id for each key user showed:

uid=1008(alex) groups=1008(alex),100(users),1005(ops_team)
uid=1010(aminat) groups=1010(aminat),100(users),1005(ops_team)
uid=1002(lateef) groups=1006(lateef),1002(dev_team)

This confirms:

- Each user retains their primary group (their username)
- Each user is also a member of their departmental supplementary group
- Group assignments match the company's Dev/QA/Sec/Ops structure

3. Privilege Boundary Enforcement

Alex (ops_team) has sudo privileges

sudo -l

Output shows commands alex is allowed to run.
This matches Part 5 of the assignment where alex was added to sudoers.

This confirms that **only one ops_team user** has administrative privileges as intended.

yaa (qa_team) is denied sudo access

su - yaa
sudo whoami

Output:

yaa is not in the sudoers file.

This verifies:

- Non-ops users **cannot** escalate privileges

- Least privilege rules are active and enforced
- Department boundaries prevent unauthorized administrative access

Observations:

1. How Linux Records and Tracks Group Membership

Linux tracks group membership primarily through two files:

- **/etc/group** → lists each group and the users who belong to it
- **/etc/passwd** → stores the primary group ID (GID) for each user

2. How Department-Based Grouping Enforces Access Control

By assigning users to department groups (e.g., dev_team, sec_team, qa_team, ops_team), Linux enforces **role-based access control (RBAC)**.

This allows:

- Developers to access dev resources only
- Security analysts to access monitoring tools
- QA testers to access testing resources
- Ops engineers to manage system-level operations

The result is **least privilege**, **clear separation of duties**, and **stronger containment** in case of credential misuse.

3. Why Service Accounts Must Use Non-Login Shells

Service accounts like **ci_runner** are created for **automation**, not human interaction.

Using a non-login shell (e.g., /usr/sbin/nologin) ensures:

- No one can log in with the service account
- Automated tasks still run normally
- Attackers cannot exploit the account to gain shell access
- The account remains isolated and secure

This reduces risk and supports **secure CI/CD pipelines**.

4. Why Home Directories May Be Retained After User Deletion

Removing a user without deleting their home directory preserves:

- Logs
- Configuration files
- Scripts and development work
- Evidence for security audits or investigations

In DevSecOps, **data retention is critical** — deleting everything immediately could remove important artifacts needed for:

- Forensics
- Root-cause analysis
- Project continuity

Keeping the home directory protects system integrity and operational traceability.

5. How Privilege Escalation Is Controlled in DevSecOps Environments

Privilege escalation is managed using:

- **sudoers policies**
- **Visudo-controlled configuration**
- **Department roles**
- **Audit logs for every elevation attempt**

Only one designated ops user was allowed to run sudo. Everyone else received a "not in the sudoers file" message.
This ensures:

- **Accountability** → all privileged actions are logged
- **Least privilege** → only the right person can escalate
- **Risk reduction** → limits the blast radius of compromised accounts